COMPUTER ORGANIZATION AND ARCHITECTURE

GROUP PROJECT

SEM2 2025/2026

DESIGNING AND ENGINEERING A COMPUTER PERFORMANCE BENCHMARK USING HIGH LEVEL TOOLS

Total Marks: 80 marks (20% of the overall grade)

PROJECT SYNOPSIS :

This group project focuses on the practical execution and evaluation of a **Computer Performance Measurement and Analysis** within a real-world computing context. Students are expecting to implement existing available free tools to diagnose microarchitectural constraints. This project requires each group member to install, configure, and isolate a distinct performance benchmarking tool selected from either a Windows suite (*Cinebench, HWiNFO64, PassMark, CrystalDiskMark*) or a Linux suite (*sysbench, htop/top, turbostat, vmstat*). Students will systematically synthesize and analyze two contrasting operational scenarios:

- ✓ **Workload A**, which captures core pipeline processing throughput, thermal throttling dynamics, and system bottlenecks, and
- ✓ **Workload B**, which evaluates advanced architectural behaviors including Single Instruction, Multiple Data (SIMD/Vector units like AVX-512 or ARM Neon) extensions and cache coherence.

The final deliverables consist of a rigorous, data-driven technical report containing workflow charts and performance comparative evaluations, supplemented by individual video demonstrations detailing data extraction methodologies.

GROUP MEMBERS :

This project should be conducted in a group of minimum 2 students and maximum 4 students.

SUBMISSIONS VIA E-LEARNING :

Submission can be done by a member/representative of the group. The submission includes:-

1. A comprehensive report (between 40 to 45 pages) in .pdf format that covers all the topics listed in the following [60 marks].

Section	Topic	Marks [60]
	Cover Page (Members particulars, Video Link for each member) **TOC** (Table of Contents)	1 1
1.0	**Introduction** Descriptions on the Selected Tools and Justifications - version number, link to download, advantages and disadvantages	8

2.0	**Tools Installation and Configuration** 2.1 Details on Installation and Configuration of Tool 1 2.2 Details on Installation and Configuration of Tool 2 2.3 Details on Installation and Configuration of Tool 3 2.4 Details on Installation and Configuration of Tool 4 - brief manual, screenshots.	12
3.0	**Performance Benchmark Analysis and Discussions** 3.1 Detailed Explanation on Workload A and Workload B. a. Tool 1 …. d. Tool 4 3.2 Flowchart of Executing the Benchmark Based on COA Performance Metrics for Workload A. a. Tool 1 …. d. Tool 4 3.3 Flowchart of Executing the Benchmark Based on COA Performance Metrics for Workload B. a. Tool 1 …. d. Tool 4 3.4 Analysis and Discussions a. Tool 1 …. d. Tool 4 - results, screenshots, performance comparison, benefits and limitations	24
4.0	**Conclusion and Reflection**	6
5.0	**References**	5
	Appendices (Tasks distribution table, group pictures, etc)	3

2. A recorded video (from each member) to demonstrate the steps on executing the performance benchmark, or how to use the selected tool to obtain the required **COA Performance Metrics** for **Workload A** and **Workload B**. Minimum duration of a video (from each member) is 6 minutes and maximum 8 minutes. [20 marks].

Contents	Marks [20]
Self Introduction	1
Demo on how to use the tool/extract the information/ execute the performance benchmark using the selected tool.	5
Demo on how to obtain the required COA Performance Metrics for: Workload A Workload B	8
Analysis and discussion	6

SUBMISSIONS DUE DATE :

30 JUNE 2026, 23:59PM

PROJECT SUMMARY:

A. Student Tasks:

Students will act as hardware and system optimization engineers to establish a controlled testing environment. They must analyze hardware performance bottlenecks by deploying specific free tools on **Windows** and on **Linux** to capture and isolate hardware behaviors across different operating system abstractions. The available free tools offers different kinds of performance benchmark that are useful to be implemented in real workloads and platforms. Students have the freedom to choose which tool to be used and report the findings.

B. Instructions:

1. Select ANY four combinations (depending on the number of members) of benchmark tools to be used in your GROUP project.

2. Each member must install and configure the selected tool. Choose based on your computer/laptop's Operating System. Each member MUST choose ONE tool and different from other members.

3. Each member performs the related benchmark for both **Workload A** and **Workload B** using the installed tool.

4. Analyze and discuss the performances/findings for both Workloads based on the given **COA Performance Metrics**. This means in a four members group report, it MUST include the analysis and discussions on four different tools.

C. The Toolkits (given 4 Tools Per Suite):

- **Windows Suite:-**

1. *Cinebench 2026 / R23* (Heavy synthetic micro-architecture pipeline stress)
2. *HWiNFO64* (Hardware sensor logging for thermal throttling and clock rates)
3. *PassMark PerformanceTest (Windows Edition)* (Component-level standard scoring)
4. *CrystalDiskMark* (Storage subsystem controller overhead monitoring)

- **Linux Suite:-**

1. sysbench (Modular command-line CPU, memory, and thread subsystem stressors)
2. htop / top (Real-time logical thread scheduler and context switch monitoring)
3. turbostat / lscpu (Direct Model-Specific Register mapping for clock frequency and power tracking)
4. vmstat (Virtual memory statistic tracker capturing page faults and CPU wait states)

D. Workload A:

Workload A specifies operational matrices detail and how students should set up their tools, execute workloads, track architectural performance parameters, and extract data to compute fundamental Computer Organization and Architecture (COA) metrics. Students need to independently refer related online references (if required) and explore the implementation of selected tool based on **How to Extract Findings (Hints)** to obtain the desired **COA Performance Metrics** for the **designed Workload** as tabulated in Table A.1 and Table A.2.

- **Windows Suite:-**

Table A.1: Windows-based Performance Measurement Architecture Matrix.

Tool	Design the Workload	COA Performance Metrics & Analysis	How to Extract Findings (Hints)
1. Cinebench R23 / 2026 [+]	Run the **30-Minute Multi-Core Custom Loop** to continuously trigger a rendering task.	*** CPU Execution Time (T_{cpu})** *** Throughput (Task-level)** *** Pipeline Saturation Efficiency**	Read the **Total Completion Time** (seconds) directly from the log and note the raw **Cinebench Points Index** (measures Multi-threaded vs. Single-threaded instruction scaling).
2. HWiNFO64	Launch in **"Sensors-Only" Mode** running silently in the background while Cinebench is active.	*** Thermal Throttling Events** *** Clock Frequency Delta ($\Delta\tau$)** *** Dynamic Power Consumption (W)**	Export the .CSV data log. Locate rows labeled "Core Thermal Throttling [Yes/No]". Calculate the gap between the chip's **Base Clock Frequency** and the downclocked frequency caused by heat generation.
3. PassMark Performance Test [+]	Execute the standard **CPU Instruction Suite** (Integer Math, Floating Point, and Sorting algorithms).	*** MIPS (Million Instructions Per Second)** *** CPI (Cycles Per Instruction Estimation)** *** ISA Structural Bottlenecks**	Click "View Test Results Log". Extract the raw structural operation scores (e.g., Million operations per second). Students use the mathematical transformation: MIPS= Operations per Second/10^6

Tool	Design the Workload	COA Performance Metrics & Analysis	How to Extract Findings (Hints)
4.CrystalDiskMark	Configure a **4KiB Block Read/Write test** with a heavy queue and thread depth (Q=32, T=1).	*** Memory Bus Controller Overhead** *** Direct Memory Access (DMA) vs. Interrupt I/O handling**	Read the primary display output table under the **RND4K** row header. Extract throughput scores measured in MB/s and Input/Output Operations Per Second (IOPS).

[+] Compatible with MacOS

- **Linux Suite:-**

Table A.2: Linux-based Performance Measurement Architecture Matrix

Tool	Design the Workload (Terminal Commands)	COA Performance Metrics & Analysis	How to Extract Findings (Hints)

1. sysbench	Execute a multi-threaded compute workload and a sequential cache memory strain:
	
	`sysbench cpu --cpu-max-prime=50000 --threads=$(nproc) run`
	
	`sysbench memory --memory-block-size=4K --memory-total-size=50G run`
		*** CPU Execution Time (T_{cpu})**
		
		*** MIPS Calculation Base**
		
		*** Memory Interface Pipeline Speed**
			Analyze the final terminal printout. Extract the **total time:** value (seconds). Locate **events per second** to determine ALU capability. For memory, copy the reported **Transfer speed** (MiB/sec).

2. htop / top [+]	Run `htop` inside a separate split-screen pane while running `sysbench` resource loops.	*** Logical Thread Scheduling Efficiency** *** Active Core Load Distribution**	Inspect the color-coded top bars to observe thread balance. Read the **Tasks:** metadata line to extract the active process states and identify thread serialization bottlenecks.

Tool	Design the Workload (Terminal Commands)	COA Performance Metrics & Analysis	How to Extract Findings (Hints)
3. turbostat	Run with administrative rights before and during processing loads: `sudo turbostat --interval 1`	*** Hardware Clock Period (τ)** *** Model-Specific Register (MSR) Core States** *** Energy-Per-Instruction metrics**	Review the output columns. Extract values from **Bzy_MHz** (Actual operating clock rate) and **PkgWatt** (Real-time processor power draw) to correlate instruction density with heat constraints.
4. vmstat	Run the virtual memory monitoring daemon on a set interval during memory tasks: `vmstat 1 10`	*** Context Switching Frequency (CS)** *** CPU Kernel Wait States (wa)** *** Page Fault Memory Latencies**	Look at the columns under the **system** header: read values under cs for **Context Switches per second**. Under the cpu header, read wa to check for cycles wasted waiting for I/O lines.

[+] **Compatible with MacOS**

E. Workload B:

Workload B provides another architectural angle that reflects the current modern computer processors, for example Single Instruction, Multiple Data (SIMD/Vector units like AVX-512 or ARM Neon) and memory hierarchy cache coherence. AVX-512 is an incredibly powerful SIMD extension designed by Intel for x86-64 architecture processors.ARM Neon is the foundational SIMD hardware engine designed for ARM architecture processors (powering smartphones, tablets, Apple Silicon Macs, and low-power IoT devices). Students need to independently refer related online references (if required) and explore the implementation of selected tool based on **How to Extract Findings (Hints)** to obtain the desired **COA Performance Metrics** for the **designed Workload** as tabulated in Table B.1 and Table B.2.

- **Windows Suite:-**

Table B.1: Windows Performance Measurement Architecture Matrix (Alternative Workloads)

Tool	Design the Workload for SIMD	COA Performance Metrics & Analysis	How to Extract Findings (Hints)
1. Cinebench R23 / 2026 +	Run the **Single-Core (1-Thread) Execution Loop** for 10 minutes instead of multi-core.	* **Single-Thread Instruction Pipeline Efficiency** * **Maximum Boost Clock Intermittent Core Shifting**	Extract the **Single-Core Score** (points). Analyze the architecture's raw clock-for-clock instruction throughput (IPC) without the interference of multi-threaded synchronization overhead.
2. HWiNFO64	Configure HWiNFO64 to log specifically the **CPU Core Utility** and **Individual Core Temperatures** during the Single-Core Cinebench loop.	* **Dynamic Thread Migration** * **Localized Thermal Hotspots (Thermal Density)**	Open the log and track which specific physical core was active. Observe how the Windows scheduler shifts a single heavy thread across different cores to prevent localized thermal throttling.
3. PassMark PerformanceTest +	Isolate and execute only the **Floating-Point Math and Vector Math** portions of the CPU test suite.	* **SIMD (Single Instruction, Multiple Data) Efficiency** * **Floating-Point Unit (FPU) Throughput**	Review the detailed advanced metrics log. Extract the **GFLOPS (Giga-Floating Point Operations Per Second)** rating to evaluate how efficiently the microarchitecture handles complex vector extensions (AVX2/AVX-512).
4. CrystalDiskMark	Run a **Sequential 64KiB Read/Write test** with no queues (SEQ64K, Q=1, T=1) to simulate loading raw uncompressed memory blocks.	* **Non-Burst System Bus Latency** * **Cache Line Fill Buffering Controller Limits**	Extract the **MB/s transfer rate**. Compare this low-queue sequential data flow against previous high-queue tests to analyze how hardware command queuing (NCQ) mitigates

Tool	Design the Workload for SIMD	COA Performance Metrics & Analysis	How to Extract Findings (Hints)
			mechanical/bus latencies.

[+] **Compatible with MacOS**

- **Linux Suite:-**

Table B.2: Linux Performance Measurement Architecture Matrix (Alternative Workloads)

Tool / Utility	Design the Workload (Terminal Commands) for SIMD	COA Performance Metrics & Analysis	How to Extract Findings (Hints)
1. sysbench	Force a heavy, localized **Threads Subsystem lock contention test** to simulate intense inter-process scheduling: `sysbench threads --thread-yields=5000 --thread-locks=8 --threads=64 run`	* **Thread Synchronization Overhead** * **Cache Coherency Latency (*L1*/*L2* Cache Invalidation)**	Read the final summary output block. Extract the **execution time** and the **latency avg** (ms). Analyze how execution time degrades when multiple threads fight for the same memory address spaces.
2. htop / top [+]	Run `htop` in a separate terminal panel, but change the display filter settings to **show individual kernel threads** (F2 → Setup → Display options → Show custom kernel threads).	* **Hardware Interrupt Distribution** * **Kernel-to-User State Switching Traps**	Observe the active thread bars. Look for high volumes of red bars (which represent time spent servicing kernel/system calls) versus green bars (user applications) during the thread-lock stress test.
3. turbostat	Run `turbostat` with a high polling frequency while running the `sysbench threads` lock loop: `sudo turbostat --interval 0.5`	* **Core Sleep State (*C-States*) Transition Latency** * **Energy-saving Throttle Backs**	Examine the **%Cool** or **%C1**, **%C6** column states. Extract data showing whether cores are prematurely entering low-power sleep modes because they are stalled waiting for memory thread locks to open up.

Tool / Utility	Design the Workload (Terminal Commands) for SIMD	COA Performance Metrics & Analysis	How to Extract Findings (Hints)
4. vmstat	Run the virtual memory daemon with a focus on system traps and context spikes: `vmstat 1 20`	*** System Interrupt Rate (*in*)** *** CPU Idle time due to Execution Stalls (*id*)**	Monitor the data streams listed under the **system** header column block. Extract the **in** (interrupts) data to determine how hard the hardware interrupt controller is being pushed by software synchronization loops.

[+] **Compatible with MacOS**

- End of page -